Learning Objectives

In the video, you saw how to implement a simple parser that generates assembly code for arithmetic expressions.

- Understand why parsing and code generation can be separated
- Represent arithmetic expressions as expression trees
- Implement a dynamic tree data structure
- Work with algebraic data types (ADTs)
- Refactor the parser to construct expression trees instead of generating assembly code
- Organize a larger program into multiple translation units

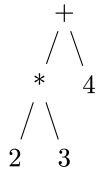
Keeping Your Work

Keep the code you develop for the compiler project in a `not-abc` directory. If you are already familiar with Git, you may instead create a `not-abc` repository and keep your work there. We will introduce Git in a later worksheet, so **no Git experience is required at this point**.

Expression Trees: What They Are and How We Want to Use Them

In the video, our parser generated assembly code directly while parsing an arithmetic expression. For example, when it encountered an integer literal, it emitted a load instruction; when it recognized an addition or multiplication, it emitted the corresponding instruction.

This is a perfectly reasonable way to implement a simple compiler. Historically, many compilers have worked in a similar way. For our compiler project, however, we will separate parsing from code generation. Instead of generating assembly code immediately, our parser will construct a data structure that represents the expression it has recognized. For example, the expression `2 * 3 + 4` can be represented by the following expression tree:



The leaves of this tree represent integer literals. The inner nodes represent operations. For now, we therefore need two kinds of nodes:

- **Primary nodes** represent integer literals and have no successors.
- **Binary nodes** represent operations such as addition and multiplication and have two successors.

As with the linked lists that you implemented earlier, the number of nodes is not known in advance. We therefore need a dynamic data structure and will work with pointers to objects of type `Expr`.

Before worrying about how `struct Expr` should look internally, let us first think about how we would like to use this data structure.
We would like to write

```
createIntegerExpr(2)
```

to create a leaf representing the integer value `2`.
Similarly,

```
createBinaryExpr(EXPR_MUL,
                 createIntegerExpr(2),
                 createIntegerExpr(3))
```

should create a binary node representing the expression `2 * 3`. And

```
createBinaryExpr(EXPR_ADD,
                 createBinaryExpr(EXPR_MUL,
                     createIntegerExpr(2),
                     createIntegerExpr(3)),
                 createIntegerExpr(4))
```

should create a binary node representing the expression `2 * 3 + 4`. Notice how the nested function calls already reflect the structure of the expression tree. The outer call creates the root node `+` of the tree. Its left successor is the node returned by the nested `createBinaryExpr` call, while its right successor is the node returned by `createIntegerExpr(4)`. In exactly the same way, the `*` node gets the nodes for `2` and `3` as its successors.

© Michael C. Lehn | CC BY-SA 4.0

In other words, evaluating these function calls constructs the tree shown above from the leaves upwards.

There are two more things we will need.

First, we would like to do something useful with an expression tree once we have constructed it. For example, we might want to visualize the tree — or, later, generate assembly code from it. We will therefore provide a function

`printExprTree(expr)`

that traverses an expression tree and produces a textual representation of it.

Second, our expression trees are dynamic data structures, just like the linked lists we worked with earlier. Nodes will be allocated dynamically, and eventually their memory has to be released again. We will therefore also need

`releaseExpr(expr)`

which releases an entire expression tree.

Before thinking about how any of these functions should be implemented, let us put the pieces together and write a small program `xtest_expr.abc` that uses the interface we would like to have:

```
@ <stdio.hdr>
@ "expr.hdr"

fn main()
{
    local expr: -> Expr;

    expr = createBinaryExpr(EXPR_ADD,
                            createBinaryExpr(EXPR_MUL,
                                             createIntegerExpr(2),
                                             createIntegerExpr(3)),
                            createIntegerExpr(4));
    printExprTree(expr);
    printf("\n");

    releaseExpr(expr);
}
```

Representing and Implementing Expression Trees

Now that we know how we would like to use expression trees, we can think about their representation and about the declarations required for this interface.

A simple representation is to use a `struct Expr` for every node of an expression tree. Since a node can represent different things, we first need a way to distinguish the different kinds of nodes. For this purpose, we introduce an enumeration `ExprKind`:

```
enum ExprKind
{
    EXPR_ADD,
    EXPR_MUL,

    EXPR_INTEGER,
};
```

A first idea is to put all information required by the two different kinds of nodes directly into `struct Expr`:

```
struct Expr
{
    kind:           ExprKind;
    left, right:    ->Expr;
    decimal:        u64;
};
```

© Michael C. Lehn | CC BY-SA 4.0

The member kind tells us how a particular node has to be interpreted. If `kind` is `EXPR_ADD` or `EXPR_MUL`, the node is a binary node and `left` and `right` point to its two successors. If `kind` is `EXPR_INTEGER`, the node is a primary node and decimal stores the integer literal.

Note that not every member is meaningful for every kind of node. For a binary node, we use `left` and `right`; for an integer node, we use decimal. The member kind tells us which interpretation is valid. With the declaration above, if `expr` is a pointer to an expression node (that is, of type `->Expr`), accesses to these members would look like:

- `expr->left`, `expr->right`,
- or `expr->decimal`.

Which of these accesses is meaningful depends on `expr->kind`. This works, but we can make the notation more descriptive — and thereby make our code easier to read and less error-prone. Ideally, an expression such as `expr->binary.left` should already tell us that we are accessing information belonging to a binary node.

We can achieve this by further structuring `Expr` with nested structs[1]:

```
struct Expr
{
    kind: ExprKind;
    binary:     struct Binary {
        left, right: ->Expr;
    };
    primary: struct Primary {
        decimal: u64;
    };
};
```

Now the corresponding accesses are:

- `expr->binary.left`, `expr->binary.right`,
- or `expr->primary.decimal`.

and the structure of the access itself documents which kind of node the member belongs to.
We now have everything needed to specify the interface used by our test program. We collect these declarations in the header file `expr.hdr`:

```
enum ExprKind
{
    EXPR_ADD,
    EXPR_MUL,

    EXPR_INTEGER,
};

struct Expr
{
    kind: ExprKind;
    binary:     struct Binary {
        left, right: ->Expr;
    };
    primary: struct Primary {
        decimal: u64;
    };
};

// constructors (caller becomes owner)
extern fn createBinaryExpr(kind: ExprKind, left: ->Expr, right: ->Expr): ->Expr;
extern fn createIntegerExpr(val: u64): ->Expr;

// destructor
extern fn releaseExpr(expr: ->Expr);

// meothods
extern fn printExprTree(expr: ->Expr);
```

[1] This organization will also make `Expr` easier to extend when we add more kinds of expression nodes later.

© Michael C. Lehn | CC BY-SA 4.0

Step 1 — Setting Up the Expression Tree Module

Create the files `expr.hdr` and `xtest_expr.abc` exactly as described above. Before implementing any of the functions, we first make sure that the interface is consistent with our test program.

1. Compile the test program without linking:

   ```
   abc xtest_expr.abc -c
   ```

   Recall, the option `-c` tells the compiler to create an object file without linking an executable. Make sure that the program compiles without errors or warnings. In particular, this is a good opportunity to catch any typos in `expr.hdr` or `xtest_expr.abc` before we start implementing anything.
2. Now create the source file `expr.abc`. Do not implement the expression tree yet. Instead, start with the following stubs — minimal implementations whose main purpose is to give us a complete program that can already be compiled and linked:

   ```
   @ <stdio.hdr>
   @ "expr.hdr"

   fn createBinaryExpr(kind: ExprKind, left: ->Expr, right: ->Expr): ->Expr
   {
       printf("not implemented\n");
       return left;
   }

   fn createIntegerExpr(val: u64): ->Expr
   {
       printf("not implemented\n");
       return nullptr;
   }

   fn releaseExpr(expr: ->Expr)
   {
       printf("not implemented\n");
   }

   fn printExprTree(expr: ->Expr)
   {
       printf("not implemented\n");
   }
   ```

   Compile and link the complete program with

   ```
   abc xtest_expr.abc expr.abc
   ```

   This should produce an executable program. Run it and verify that the stub functions are actually called. At this point, our program does not do anything useful yet — but we already know that the translation units fit together and that we can now implement and test one function at a time.
3. As a first function, implement `createIntegerExpr`. Compile and run the program again immediately afterwards. There is still not much to see, but this allows us to catch mistakes early. You will also notice that malloc requires us to include `stdlib.hdr` ;-).

   ```
   fn createIntegerExpr(val: u64): ->Expr
   {
       local n: -> Expr = malloc(sizeof(*n));
       assert(n);
       n->kind = EXPR_INTEGER;
       n->primary.decimal = val;
       return n;
   }
   ```

4. Now that we can at least create one kind of node, it is useful to implement `printExprTree` next. This will finally allow us to see that our program is actually doing something.

© Michael C. Lehn | CC BY-SA 4.0

You can use the following implementation. Its particular output format will also allow us to visualize the expression tree later:

```
fn printExprTree(expr: ->Expr)
{
    if (expr->kind == EXPR_INTEGER) {
        printf("[%llu]", expr->primary.decimal);
    } else if (expr->kind >= EXPR_ADD && expr->kind <= EXPR_MUL) {
        printf("[ ");
        switch (expr->kind) {
            case EXPR_ADD: printf("+ "); break;
            case EXPR_MUL: printf("* "); break;
            default: assert(0);
        }
        printExprTree(expr->binary.left);
        printf(" ");
        printExprTree(expr->binary.right);
        printf("] ");
    }
}
```

The idea behind this representation is simple:

- For a primary node representing, for example, the integer value `42`, we print `[42]`.
- For a binary node, we first print `[` and the operator (`+` or `*`), followed by the left and right successor, and finally `]`.

This produces output such as `[ * [2] [3]]` or `[+ [ * [2] [3]] [4]]`.

5. Now implement `createBinaryExpr` yourself. Remember the structure of `Expr` and how `createIntegerExpr` allocates and initializes a node.
   If everything works, the test program should now produce
   ```
   [ + [ * [2] [3]] [4]]
   not implemented
   ```
6. Before implementing `releaseExpr`, let us verify that our data structure really represents the tree we intended to construct. The output produced by `printExprTree` was deliberately chosen such that it can be processed directly by the LaTeX package `forest`.
   Create a file `tree_header.tex` containing
   ```
   \documentclass[preview, margin=0.2cm]{standalone}
   \usepackage{forest}
   \begin{document}
   \begin{forest}
   ```
   and a file `tree_footer.tex` containing
   ```
   \end{forest}
   \end{document}
   ```
   We can now combine the header, the output of our program, and the footer into a complete LaTeX document:
   ```
   (cat tree_header.tex; ./a.out; cat tree_footer.tex) > tree.tex
   ```
   Inspect `tree.tex`. You should be able to recognize the textual tree produced by calling `printExprTree` between the LaTeX header and footer.
   If LaTeX is installed, generate the visualization with
   ```
   lualatex tree.tex
   ```
   Open `tree.pdf` and compare it with the expression tree at the beginning of this worksheet.
7. There is one important function left: `releaseExpr`. Every node created with `malloc` eventually has to be released. Implement `releaseExpr` recursively. For a binary node, make sure that both successor trees are released before releasing the node itself. A primary node has no successors and can therefore be released immediately.
   The recursive structure is very similar to `printExprTree`: instead of printing the nodes while traversing the tree, we now release them.
   Once `releaseExpr` is implemented, the not implemented message should disappear and our first implementation of expression trees is complete.

© Michael C. Lehn | CC BY-SA 4.0

Thinking One Step Ahead

Before moving on, we will make one small change to our expression trees. The change itself is technically simple. The difficult part is realizing that it is a useful change to make now.

Our next goal is to modify the parser from the videos such that it constructs expression trees instead of generating assembly code. The parser receives tokens from the lexer, and for an integer literal the character sequence recognized by the lexer is already available as a unique string. If our expression tree can use this representation directly, the parser will become simpler.

Coming up with such a design decision requires a little foresight: we have to think about how the data structure will be used later and work backwards from there. This is not something you should expect to get right immediately.

Programming is much like mathematics in this respect. Experience gradually develops into intuition about which representation or approach might make the next step easier. You build this intuition by looking at how other people solve problems, understanding why their solutions work, and then trying many things yourself.

The next step therefore contains a small design decision that I would not necessarily expect you to come up with on your own yet. Over time, decisions like this will become increasingly natural.

Step 2 — Using Unique Strings for Integer Literals

After the philosophical prelude above, let us simply assume that, for good reasons, we have decided to change the declaration of Expr to the following:

```
struct Expr
{
    kind: ExprKind;
    binary:     struct Binary {
        left, right: ->Expr;
    };
    primary: struct Primary {
        decimal: ->UStr;
    };
};
```

Tasks

We will not question this design decision any further for now. Instead, let us deal with the trail of technical consequences that follows from it.

1. Change the declaration of `Expr` in `expr.hdr` as shown above. Then try to compile the program again:

   ```
   abc xtest_expr.abc expr.abc
   ```

   The compiler will immediately point out one consequence of our change: `expr.hdr` now uses the type `UStr` and therefore has to include `ustr.hdr`.

   You could now continue playing the game *“compile, read the error message, fix the problem”* until everything works again. This is a perfectly valid strategy. It becomes more fun, however, if we can anticipate some of the necessary changes before the compiler tells us about them.
2. `createIntegerExpr` should now expect a parameter of type `->UStr` instead of `u64`. This has at least two immediate consequences that we can fix in advance:
   - In `xtest_expr.abc`, change calls such as

     ```
     createIntegerExpr(4)
     ```

     to

     ```
     createIntegerExpr(UStrCreate("4"))
     ```

     Do this for all integer nodes created by the test program.
   - Adapt the implementation of `createIntegerExpr` in `expr.abc` accordingly.
3. Try to compile again with: `abc xtest_expr.abc expr.abc`

   If you fixed all compilation errors in the previous task, you should now encounter a different kind of error. Look carefully at the diagnostic. This time the individual translation units can be compiled, but an executable program still cannot be created.
   - Is this a compiler error or a linker error? What is missing?
   - Once you have identified the problem, build the complete program with

     ```
     abc xtest_expr.abc expr.abc ustr.abc
     ```

     Now everything required to create the executable is present.

© Michael C. Lehn | CC BY-SA 4.0

4. Run the program. It should compile and link now — but the output for the integer nodes looks rather strange. Think about what has changed: `expr->primary.decimal` is no longer an integer value. It is now a pointer to a unique string. Find the corresponding code in function `printExprTree` and adapt the `printf` call. Only the format string has to change:

   ```
   printf("[%s]", expr->primary.decimal);
   ```

   Compile and run the program again. The textual representation of the expression tree should now look exactly as it did before our change.

We have changed the internal representation of integer nodes without changing what our expression tree does from the outside. In the next step, we will finally make use of this new representation when connecting our expression trees to the parser.

Step 3 — Letting the Parser Build Expression Trees

We are now ready to return to the parser from the video. Its basic structure will remain almost unchanged, but we will make two important modifications:

- We separate the parser for expressions into a header file `parseexpr.hdr`, which describes its public interface, and a translation unit `parseexpr.abc`, which contains its implementation.
- Instead of generating assembly code while parsing, the parser will construct and return an expression tree.

Let us again start with the interface before thinking about its implementation. Other translation units only need one function for parsing an expression. Create `parseexpr.hdr` with the following content:

```
@ "expr.hdr"

extern fn parseExpr(): ->Expr;
```

So `parseExpr` no longer returns the number of a register containing the result, as it did in the video. Instead, it returns a pointer to the root node of the expression tree that represents the parsed expression.

And again, before thinking about how to implement `parseExpr`, let us first look at how we would like to use and test the parser. Create the following program `xtest_parseexpr.abc`:

```
@ <stdio.hdr>
@ "lexer.hdr"
@ "parseexpr.hdr"

fn main()
{
    getToken();
    local expr: ->Expr = parseExpr();
    if (expr && token.kind == EOI) {
        printExprTree(expr);
        printf("\n");
        releaseExpr(expr);
    }
}
```

This should look familiar from the parser in the video — and deliberately so. We still obtain the first token with `getToken()` and then ask `parseExpr()` to parse an expression.
The important difference is what a successful call to `parseExpr()` gives us. A `nullptr` indicates that no expression could be parsed. Otherwise, the returned pointer refers to the root of the expression tree that was constructed while parsing.
We additionally check that `token.kind == EOI`. This ensures that the parser did not merely recognize an expression at the beginning of the input, but that the complete input was consumed. If both conditions are satisfied, we print the resulting expression tree.

The grammar itself remains exactly as in the video:

```
expr   = term { "+" term }
term   = factor { "*" factor }
factor = decimal-literal
       | "(" expr ")"
```

Following the same convention as in the video, the three grammar rules will be implemented by three corresponding parsing functions: `parseExpr`, `parseTerm`, and `parseFactor`.

© Michael C. Lehn | CC BY-SA 4.0

The implementation of `parseexpr.abc` that we will use is closely based on the parser developed in the video:

```
@ <stdio.hdr>
@ <stdlib.hdr>
@ "lexer.hdr"
@ “parseexpr.hdr"

fn syntaxError(msg: ->readonly char)
{
    printf("error: %s\n", msg);
    exit(1);
}

fn parseFactor(): ->Expr;
fn parseTerm(): ->Expr;

fn parseExpr(): ->Expr
{
    local left: ->Expr = parseTerm();
    if (!left) {
        return nullptr;
    }
    while (token.kind == PLUS) {
        getToken();
        local right: ->Expr = parseTerm();
        if (!right) {
            syntaxError("term expected");
        }
        left = createBinaryExpr(EXPR_ADD, left, right);
    }
    return left;
}

fn parseTerm(): ->Expr
{
    local left: ->Expr = parseFactor();
    if (!left) {
        return nullptr;
    }
    while (token.kind == ASTERISK) {
        getToken();
        local right: ->Expr = parseFactor();
        if (!right) {
            syntaxError("factor expected");
        }
        left = createBinaryExpr(EXPR_MUL, left, right);
    }
    return left;
}

fn parseFactor(): ->Expr
{
    if (token.kind == DECIMAL_LITERAL) {
        local expr: ->Expr = createIntegerExpr(token.val);
        getToken();
        return expr;
    } else if (token.kind == LPAREN) {
        getToken();
        local expr: ->Expr = parseExpr();
        if (!expr) {
            syntaxError("expression expected");
        }
        if (token.kind != RPAREN) {
            syntaxError("')' expected");
        }
        getToken();
        return expr;
    } else {
        return nullptr;
    }
}
```

© Michael C. Lehn | CC BY-SA 4.0

Tasks

1. Create `parseexpr.hdr`, `parseexpr.abc`, and `xtest_parseexpr.abc` as shown above. Alternatively, if you implemented the parser while watching the videos, adapt your existing code accordingly. Compile the test program with

   ```
   abc xtest_parseexpr.abc parseexpr.abc lexer.abc ustr.abc expr.abc
   ```

   By now, typing the command for building our program is becoming almost as tedious as writing the program itself. We will fix this soon by using a **Makefile**.

   Test the parser with several arithmetic expressions. Try both expressions that satisfy our grammar and expressions that do not. In particular, make sure that your tests cover addition, multiplication, parentheses, and operator precedence.

   Finally, as in Step 1, Task 6, use the output of `printExprTree` together with `lualatex` to generate PDFs of some of the expression trees. Check that the visualized trees actually represent the expressions you entered.
2. Explain to your neighbor why the forward declarations of `parseFactor` and `parseTerm` in lines 12 and 13 are necessary. As a related question, make sure that you understand the difference — and the similarity — between a function declaration and a function definition. This is something I might ask in an oral exam.
3. Explain the purpose of

   ```
   left = createBinaryExpr(EXPR_ADD, left, right);
   ```

   in parseExpr. What tree is constructed, for example, while parsing `2 + 3 + 4`. Draw the resulting expression tree and compare it with the tree produced by your test program.

   Also consider and what the tree looks like for a longer expression as `2 + 3 + 4 + 5 + 6`. Keep this structure in mind. It will become relevant later when we discuss the associativity of operators.

Step 4 — From Token Kinds to Expression Kinds

Our lexer and our expression trees use different enumerations to describe operators. For example, the lexer represents `+` by the token kind `PLUS`, while an expression tree represents the same operator by `EXPR_ADD`.

Add the following helper function to `parseexpr.abc`:

```
fn exprKindFromToken(token: Token): ExprKind
{
    switch (token.kind) {
        case PLUS: return EXPR_ADD;
        case ASTERISK: return EXPR_MUL;
        default:
            printf("token: %d (%s)\n", token.kind, tokenKindStr(token.kind));
            assert(0 && "internal error: exprKindFromToken");
            return 0;
    }
}
```

This function is an implementation detail of the parser. Therefore defined only `parseexpr.abc` and not declared in `parseexpr.hdr`.

Task

Find the appropriate places in `parseexpr.abc` where `exprKindFromToken` can be used and modify the parser accordingly. Compile and test your program again afterwards.

Looking Ahead

Before moving on to the final step, take another look at the implementations of `parseExpr` and `parseTerm`. Do not worry about changing anything yet — just compare their structure carefully. They are not merely similar. Structurally, they are essentially the same function now.
In the final step, we will take advantage of this observation and combine their implementations into a single function. The benefit will not only be fewer lines of code. More importantly, it will make our parser much easier to extend with additional binary operators such as `-`, `/`, `%`, and others.

© Michael C. Lehn | CC BY-SA 4.0

Step 5 — Towards a More General Parser

1. Add the following internal helper function to `parseexpr.abc`:

```
fn tokenKindPrec(kind: TokenKind): int
{
    switch (kind) {
        default: return 0;
        case ASTERISK: return 13;
        case PLUS: return 12;
    }
}
```

The function assigns a precedence to the token kinds representing our binary operators. We will discuss the meaning of these precedence values in more detail shortly.
In `parseExpr`, replace

```
while (token.kind == PLUS) {
```

with

```
while (tokenKindPrec(token.kind) == 12) {
```

and in `parseTerm`, replace

```
while (token.kind == ASTERISK) {
```

with

```
while (tokenKindPrec(token.kind) == 13) {
```

Compile and test your parser again. Its behavior should not have changed.
Now compare `parseExpr` and `parseTerm` once more. If you did not see the structural similarity before, have another look. ;-)

2. Now replace the previous implementations of `parseExpr` and `parseTerm`, together with their forward declarations, by:

```
fn parseBinary(prec: int): ->Expr;
fn parseFactor(): ->Expr;

fn parseExpr(): ->Expr
{
    return parseBinary(1);
}

fn parseBinary(prec: int): ->Expr
{
    local left: ->Expr = parseFactor();
    if (!left) {
        return nullptr;
    }
    for (local p: int = tokenKindPrec(token.kind); p >= prec; --p) {
        while (tokenKindPrec(token.kind) == p) {
            local tok: Token = token;
            getToken();
            local right: ->Expr = parseBinary(p + 1);
            if (!right) {
                syntaxError("expression expected\n");
            }
            left = createBinaryExpr(exprKindFromToken(tok), left, right);
        }
    }
    return left;
}
```

Compile and test your parser again, using the same expressions as before. The resulting expression trees should be unchanged.
Now take some time to let this code sink in. Walk through `2 + 3 * 4`, starting with function `parseBinary(1)`, and compare what happens with the previous implementation.
You should discover that we are essentially doing the same thing as before: higher-precedence operators are processed first. Previously, precedence was encoded in the structure of functions `parseExpr` and `parseTerm`. Now it is represented by a number, allowing the same recursive function to handle both levels.

3. Describe the grammar that is now implemented by our parser.

© Michael C. Lehn | CC BY-SA 4.0